

Punto 5.8 — Migraciones en equipos: conflictos y buenas prácticas

Práctica

Ejercicio: Simular un conflicto de migraciones en el proyecto AceriaData. Crear dos ramas, generar una migración en cada una y resolver la divergencia de forma coherente con el snapshot. Una migración propia que aún sea local, no compartida y no aplicada puede retirarse y regenerarse después de incorporar la migración del compañero. Una migración ya compartida o aplicada no se resuelve renombrando archivos ni borrándola sin coordinación. Revisar las migraciones en un pull request simulado y validar el estado final del modelo.

Contexto del proyecto: En el punto 5.7 se estudiaron las migraciones idempotentes y los scripts SQL, incluyendo su generación, revisión y aplicación en pipelines de CI/CD. En este punto se profundiza en las migraciones en equipos, incluyendo los conflictos, los merges y las buenas prácticas. Esta técnica se usará en el punto 5.9 para el patrón Repositorio y Unidad de Trabajo en aplicaciones empresariales.

Paso 1: Abrir el proyecto

```
cd AceriaData
git status
```

cd AceriaData → entra en la carpeta raíz del proyecto. git status → muestra el estado del repositorio.

Resultado esperado: el repositorio está limpio y en la rama principal.

Error común: si hay cambios sin confirmar, se deben confirmar o descartar antes de continuar.

Paso 2: Crear la rama feature/AddDetalleOrden

```
git checkout -b feature/AddDetalleOrden
```

git checkout -b → crea una nueva rama y se cambia a ella. feature/AddDetalleOrden → nombre de la rama.

Resultado esperado: se crea la rama feature/AddDetalleOrden.

Paso 3: Añadir la entidad DetalleOrden en la rama feature

Crear el archivo src/AceriaData.Domain/Entities/DetalleOrden.cs:

```
namespace AceriaData.Domain.Entities;

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    public string? Notas { get; set; }

    public virtual OrdenFabricacion Orden { get; set; } = null!;
}
```

Línea 1: namespace AceriaData.Domain.Entities; → declara el espacio de nombres.

Línea 3: public class DetalleOrden → declara la entidad.

Línea 5: public int Id { get; set; } → clave primaria.

Línea 6: `public int OrdenId { get; set; } → clave foránea.`

Línea 7: `public string ComposicionQuimica { get; set; } = string.Empty; → composición química.`

Línea 8: `public double TemperaturaColada { get; set; } → temperatura de la colada.`

Línea 9: `public string? Notas { get; set; } → notas opcionales.`

Línea 11: `public virtual OrdenFabricacion Orden { get; set; } = null!; → propiedad de navegación.`

Error común: si se olvida el namespace, el código no compila.

■ Paso 4: Añadir el DbSet de DetalleOrden al DbContext

Modificar `src/AceriaData.Infrastructure/Persistence/AceriaDbContext.cs`:

```
public DbSet<DetalleOrden> DetallesOrden { get; set; } = null!;
```

Línea 1: `public DbSet DetallesOrden { get; set; } = null!; → expone la tabla DetallesOrden.`

Error común: si no se añade el DbSet, la entidad no se incluye en el modelo y no se crea la tabla.

■ Paso 5: Generar la migración en la rama feature

```
dotnet ef migrations add AddDetalleOrden --startup-project src/AceriaData.Console
```

`dotnet ef migrations add` → genera una nueva migración. `AddDetalleOrden` → nombre de la migración. `--startup-project src/AceriaData.Console` → indica el proyecto de inicio.

Resultado esperado: se genera la migración `AddDetalleOrden` en la rama feature.

Error común: si la migración está vacía, se debe verificar que la entidad `DetalleOrden` esté declarada en el DbContext.

■ Paso 6: Hacer commit en la rama feature

```
git add .
git commit -m "Add DetalleOrden"
```

`git add .` → añade todos los cambios al área de preparación. `git commit -m` → hace commit con el mensaje indicado.

Resultado esperado: los cambios se han confirmado en la rama feature.

■ Paso 7: Volver a la rama principal

```
git checkout main
```

`git checkout main` → cambia a la rama principal.

Resultado esperado: se cambia a la rama principal.

■ Paso 8: Crear la rama bugfix/AddIndiceNumeroOrden

```
git checkout -b bugfix/AddIndiceNumeroOrden
```

`git checkout -b` → crea una nueva rama y se cambia a ella. `bugfix/AddIndiceNumeroOrden` → nombre de la rama.

Resultado esperado: se crea la rama `bugfix/AddIndiceNumeroOrden`.

Paso 9: Añadir un índice en la rama bugfix

Modificar `src/AceriaData.Infrastructure/Persistence/Configurations/OrdenFabricacionConfiguration.cs` para añadir un índice:

```
public void Configure(EntityTypeBuilder<OrdenFabricacion> builder)
{
    // ... configuración existente ...

    builder.HasIndex(o => o.NumeroOrden)
        .IsUnique()
        .HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");
}
```

Línea 1: `public void Configure(EntityTypeBuilder builder)` → método de configuración.

Línea 3: `// ... configuración existente ...` → comentario que indica que se mantiene la configuración anterior.

Línea 5: `builder.HasIndex(o => o.NumeroOrden)` → crea un índice sobre `NumeroOrden`.

Línea 6: `.IsUnique()` → marca el índice como único.

Línea 7: `.HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");` → establece el nombre del índice.

Error común: si el índice ya existe, la migración falla. Se debe verificar antes.

Paso 10: Generar la migración en la rama bugfix

```
dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console
```

`dotnet ef migrations add` → genera una nueva migración. `AddIndiceNumeroOrden` → nombre de la migración. `--startup-project src/AceriaData.Console` → indica el proyecto de inicio.

Resultado esperado: se genera la migración `AddIndiceNumeroOrden` en la rama `bugfix`.

Error común: si la migración está vacía, se debe verificar que el índice esté configurado correctamente.

Paso 11: Hacer commit en la rama bugfix

```
git add .
git commit -m "Add índice único en NumeroOrden"
```

`git add .` → añade todos los cambios al área de preparación. `git commit -m` → hace commit con el mensaje indicado.

Resultado esperado: los cambios se han confirmado en la rama `bugfix`.

Paso 12: Volver a la rama principal y hacer merge de feature

```
git checkout main
git merge feature/AddDetalleOrden
```

`git checkout main` → cambia a la rama principal. `git merge feature/AddDetalleOrden` → hace merge de la rama `feature`.

Resultado esperado: la migración `AddDetalleOrden` se integra en la rama principal.

Paso 13: Hacer merge de la rama bugfix

```
git merge bugfix/AddIndiceNumeroOrden
```

git merge bugfix/AddIndiceNumeroOrden → hace merge de la rama bugfix.

Resultado esperado: se produce un conflicto en el archivo `AceriaDbContextModelSnapshot.cs` porque ambas ramas han modificado el snapshot.

La salida del comando muestra información como la siguiente:

```
Auto-merging Migrations/AceriaDbContextModelSnapshot.cs
CONFLICT (content): Merge conflict in Migrations/AceriaDbContextModelSnapshot.cs
Automatic merge failed; fix conflicts and then commit the result.
Observaciones: el conflicto se produce en el snapshot porque ambas ramas han añadido entidades o índices al modelo.
```

Paso 14: Detectar el conflicto con git status

```
git status
```

git status → muestra el estado del repositorio.

Resultado esperado: aparece el archivo `AceriaDbContextModelSnapshot.cs` con la marca `both modified`.

La salida del comando muestra información como la siguiente:

```
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   Migrations/AceriaDbContextModelSnapshot.cs
Observaciones: el archivo del snapshot está en conflicto porque ambas ramas han modificado el modelo.
```

Paso 15: Resolver el conflicto con regeneración

Para resolver el conflicto con regeneración, se elimina la migración conflictiva, se descarta el conflicto del snapshot y se regenera la migración.

```
# 1. Abortar el merge para empezar de nuevo
git merge --abort

# 2. Hacer merge de feature
git merge feature/AddDetalleOrden

# 3. Eliminar la migración de bugfix
git checkout bugfix/AddIndiceNumeroOrden
dotnet ef migrations remove --startup-project src/AceriaData.Console
git commit -am "Eliminar migración AddIndiceNumeroOrden"

# 4. Volver a main y hacer merge de bugfix
git checkout main
git merge bugfix/AddIndiceNumeroOrden

# 5. Regenerar la migración en main
dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console
git commit -am "Add índice único en NumeroOrden"
```

Línea 1: `git merge --abort` → aborta el merge en curso.

Línea 2: `git merge feature/AddDetalleOrden` → hace merge de feature.

Línea 3: `git checkout bugfix/AddIndiceNumeroOrden` → cambia a la rama bugfix.

Línea 4: `dotnet ef migrations remove --startup-project src/AceriaData.Console` → elimina la migración conflictiva.

Línea 5: `git commit -am "Eliminar migración AddIndiceNumeroOrden"` → confirma la eliminación.

Línea 6: `git checkout main` → cambia a la rama principal.

Línea 7: `git merge bugfix/AddIndiceNumeroOrden` → hace merge de bugfix.

Línea 8: `dotnet ef migrations add AddIndiceNumeroOrden --startup-project src/AceriaData.Console` → regenera la migración en main.

Línea 9: `git commit -am "Add índice único en NumeroOrden"` → confirma la migración.

Resultado esperado: el conflicto se resuelve y la migración se regenera en la rama principal.

Error común: si se olvida eliminar la migración conflictiva, el conflicto persiste.

| Paso 16: Aplicar las migraciones

```
dotnet ef database update --startup-project src/AceriaData.Console
```

`dotnet ef database update` → aplica las migraciones pendientes.

Resultado esperado: la base de datos queda sincronizada con el modelo.

| Paso 17: Verificar las migraciones

```
dotnet ef migrations list --startup-project src/AceriaData.Console
```

`dotnet ef migrations list` → lista todas las migraciones.

Resultado esperado: aparecen `InitialCreate`, `AddDetalleOrden` y `AddIndiceNumeroOrden` con la marca (Applied).

| Paso 18: Verificar la resolución del conflicto sin rankings temporales

No uses tiempos prefijados para decidir qué estrategia es correcta. Valida el estado de Git, la cadena de migraciones y el snapshot resultante.

```
git status
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/
AceriaData.Console
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console
```

El criterio de éxito es que el árbol quede coherente, que las migraciones representen el modelo fusionado y que `has-pending-model-changes` no detecte cambios pendientes.

| Paso 19: Crear un checklist de revisión de migraciones

Crear el archivo `checklist_migraciones.md`:

markdown

Checklist de revisión de migraciones

Antes del merge

1. ¿El nombre de la migración es descriptivo?
2. ¿Los cambios son los esperados?
3. ¿Hay operaciones destructivas (DROP COLUMN, DROP TABLE)?
4. ¿El snapshot es coherente con el modelo?
5. ¿Las migraciones se aplican sin errores?
6. ¿Hay conflictos con la rama principal?

Después del merge

1. ¿Se han regenerado las migraciones conflictivas?
2. ¿Se ha aplicado la migración a la base de datos?
3. ¿Se ha verificado el estado de la base de datos?
4. ¿Se ha actualizado el script idempotente?
5. ¿Se ha actualizado el pipeline de CI/CD? Línea 1: # Checklist de revisión de migraciones → título del checklist.

Línea 3: ## Antes del merge → sección de comprobaciones antes del merge.

Línea 5: 1. ¿El nombre de la migración es descriptivo? → primera comprobación.

Línea 6: 2. ¿Los cambios son los esperados? → segunda comprobación.

Línea 7: 3. ¿Hay operaciones destructivas (DROP COLUMN, DROP TABLE)? → tercera comprobación.

Línea 8: 4. ¿El snapshot es coherente con el modelo? → cuarta comprobación.

Línea 9: 5. ¿Las migraciones se aplican sin errores? → quinta comprobación.

Línea 10: 6. ¿Hay conflictos con la rama principal? → sexta comprobación.

Línea 12: ## Después del merge → sección de comprobaciones después del merge.

Línea 14: 1. ¿Se han regenerado las migraciones conflictivas? → primera comprobación.

Línea 15: 2. ¿Se ha aplicado la migración a la base de datos? → segunda comprobación.

Línea 16: 3. ¿Se ha verificado el estado de la base de datos? → tercera comprobación.

Línea 17: 4. ¿Se ha actualizado el script idempotente? → cuarta comprobación.

Línea 18: 5. ¿Se ha actualizado el pipeline de CI/CD? → quinta comprobación.

Error común: si no se documenta el checklist, los desarrolladores pueden olvidar alguna comprobación. Se debe documentar y compartir en el equipo.

▮ Paso 20: Crear un pipeline de verificación de migraciones

Crear el archivo .github/workflows/verify-migrations.yml:

```
yaml name: Verify Migrations
```

```
on: pull_request: branches: - main
```

```
jobs: verify: runs-on: ubuntu-latest
```

```

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup .NET
    uses: actions/setup-dotnet@v4
    with:
      dotnet-version: '8.x'

  - name: Install dotnet-ef
    run: dotnet tool install --global dotnet-ef

  - name: Verify migrations are up to date
    run: dotnet ef migrations list --startup-project src/AceriaData.Console

  - name: Generate idempotent script
    run: dotnet ef migrations script --idempotent --startup-project src/AceriaData.Console --output
    migraciones.sql

  - name: Apply script to test database
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB_Test -E -i migraciones.sql

  - name: Verify database schema
    run: sqlcmd -S "(localdb)\MSSQLLocalDB" -d AceriaDB_Test -Q "SELECT COUNT(*) FROM
    [__EFMigrationsHistory];"

```

Línea 1: name: Verify Migrations → nombre del workflow.

Línea 3: on: → define el trigger.

Línea 4: pull_request: → se ejecuta en pull request.

Línea 5: branches: → ramas que activan el workflow.

Línea 6: - main → la rama principal.

Línea 8: jobs: → define los jobs.

Línea 9: verify: → nombre del job.

Línea 10: runs-on: ubuntu-latest → sistema operativo.

Línea 12: steps: → define los pasos.

Línea 13: - name: Checkout → primer paso.

Línea 14: uses: actions/checkout@v4 → acción de checkout.

Línea 16: - name: Setup .NET → segundo paso.

Línea 17: uses: actions/setup-dotnet@v4 → acción de instalación.

Línea 18: with: → parámetros.

Línea 19: dotnet-version: '8.x' → versión.

Línea 21: - name: Install dotnet-ef → tercer paso.

Línea 22: run: dotnet tool install --global dotnet-ef → instala la herramienta.

Línea 24: - name: Verify migrations are up to date → cuarto paso.

Línea 25: run: dotnet ef migrations list ... → lista las migraciones.

Línea 27: - name: Generate idempotent script → quinto paso.

Línea 28: run: dotnet ef migrations script --idempotent ... → genera el script.

Línea 30: - name: Apply script to test database → sexto paso.

Línea 31: run: sqlcmd ... -i migraciones.sql → aplica el script.

Línea 33: - name: Verify database schema → séptimo paso.

Línea 34: run: sqlcmd ... -Q "SELECT COUNT(*) FROM [__EFMigrationsHistory];" → verifica el esquema.

Error común: si el pipeline no verifica las migraciones, los conflictos se detectan en producción. Se debe verificar en el pull request.

Paso 21: Ejecutar el proyecto

```
cd ../AceriaData.Console
dotnet run
```

cd ../AceriaData.Console → entra en la carpeta del proyecto de consola. dotnet run → compila y ejecuta el proyecto.

Resultado esperado: el proyecto compila y se ejecuta sin errores.

Paso 22: Analizar la salida

La salida del proyecto muestra que las migraciones se han aplicado correctamente. La base de datos contiene las tablas OrdenesFabricacion, DetallesOrden y los índices correspondientes.

La verificación con dotnet ef migrations list muestra:

```
20240115120000_InitialCreate
20240118120000_AddDetalleOrden
20240119120000_AddIndiceNumeroOrden
Las tres migraciones están aplicadas.
```

Observaciones: el conflicto se resolvió con regeneración. La migración AddIndiceNumeroOrden se regeneró en la rama principal después del merge. El snapshot es coherente con el modelo.

Paso 23: Diagnosticar un error común

Intentar modificar una migración ya aplicada:

```
# Modificar el archivo de la migración InitialCreate
```

Resultado esperado: la base de datos queda desincronizada con el modelo. Al generar una nueva migración, EF Core no detecta el cambio porque el snapshot ya está actualizado.

Solución: revertir el cambio en la migración y generar una nueva migración para los cambios.

```
dotnet ef migrations add AddNuevoCambio --startup-project src/AceriaData.Console
```

Resultado esperado con la solución: la nueva migración refleja los cambios.

Errores comunes del ejercicio

Error Causa Solución Conflicto en el snapshot Dos ramas modificaron el snapshot Eliminar la migración conflictiva y regenerarla Migración divergente Dos ramas generaron cambios sobre snapshots distintos Integrar el cambio compartido y regenerar la migración local cuando sea seguro Migración modificada después de aplicada Se modificó el archivo de la migración Generar una nueva migración Base de datos desincronizada No se aplicaron las migraciones después del merge Aplicar dotnet ef database update Snapshot desactualizado Se modificó manualmente Regenerar con dotnet ef migrations add Migración vacía El modelo no cambió Verificar que las entidades están declaradas Rama acumulada No se eliminaron las ramas de prueba Eliminar con git branch -D

Reto resuelto: Resolver migraciones divergentes retirando y regenerando la migración propia

Reto: simular dos ramas que modifican el modelo. La migración de la rama propia todavía no se ha compartido ni aplicado a una base común. Incorpora primero el cambio del compañero y regenera después tu migración sobre el snapshot fusionado.

Paso 1: Crear dos ramas con cambios de modelo distintos

Genera una migración independiente en cada rama y confirma cada cambio. No renombres manualmente IDs, clases, atributos y snapshots para “ordenar fechas”.

Paso 2: Incorporar la migración del compañero

Vuelve a la rama que contiene tu trabajo, integra la migración del compañero y resuelve únicamente los conflictos de modelo que correspondan.

Paso 3: Retirar la migración propia si todavía es local

```
dotnet ef migrations remove --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Este paso solo es válido cuando esa migración propia no ha sido compartida ni aplicada a una base común.

Paso 4: Regenerar la migración propia sobre el snapshot fusionado

```
dotnet ef migrations add MiCambioRegenerado --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Paso 5: Validar el resultado

```
dotnet ef migrations list --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-project src/AceriaData.Console
```

Si una migración ya se compartió o se aplicó a una base común, **no se elimina simplemente el archivo**. En ese caso se necesita una migración correctiva o una coordinación explícita de rollback/despliegue.

Corrección técnica aplicada: renombrar archivos y metadatos de una migración divergente no es la estrategia docente recomendada para reconciliar snapshots.

Analogía final

Las migraciones en equipos en una acería son como las reformas coordinadas de la planta. Cuando varios equipos trabajan en la misma planta, cada uno hace su reforma. Si dos equipos reforman la misma zona, se produce un conflicto.

La solución es coordinar las reformas, eliminar una de ellas y volver a hacerla después de la otra. Las buenas prácticas son como las normas de coordinación: nombres descriptivos, sincronización frecuente, no modificar reformas ya hechas y revisar antes de aplicar.

El snapshot del modelo es como el plano actual de la planta: refleja el estado después de todas las reformas. Si dos equipos modifican el plano a la vez, se produce un conflicto. La solución es regenerar el plano después de coordinar las reformas.

El checklist de revisión es como la inspección antes de aprobar la reforma. El pipeline de verificación es como la cadena de montaje que verifica automáticamente que la reforma es correcta.

El impacto de los conflictos en el tiempo de resolución es significativo. No se establece un ranking por tiempos prefijados. La decisión depende del estado de la migración y de si ya se compartió o aplicó.

Así funcionan las migraciones en equipos: se coordinan, se sincronizan y se regeneran cuando es necesario.

| Resultado esperado

Al final del ejercicio, deberías haber:

Simulado un conflicto de migraciones en dos ramas.

Resuelto el conflicto eliminando una migración y regenerándola.

Resuelto el conflicto mediante integración y regeneración segura.

Aplicado las migraciones después del merge.

Verificado las migraciones con `dotnet ef migrations list`.

Documentado el proceso de resolución del conflicto sin usar tiempos prefijados como criterio de corrección.

Creado un checklist de revisión de migraciones.

Creado un pipeline de verificación de migraciones.

Diagnosticado el error de modificar una migración ya aplicada.

Resuelto un conflicto de migraciones sin renombrado manual de metadatos.

| Conexión con el siguiente punto

En este punto se han estudiado las migraciones en equipos, incluyendo los conflictos, las estrategias de resolución, el papel del snapshot del modelo, las buenas prácticas, la integración con revisión de código y la medición del impacto de los conflictos. Se han distinguido conflictos de modelo, migración y snapshot. Retirar y regenerar la migración propia es una estrategia válida cuando esa migración sigue siendo local, no compartida y no aplicada; una migración ya compartida o aplicada requiere una migración correctiva o un despliegue coordinado.

En el siguiente punto se estudiará el Patrón Repositorio y la Unidad de Trabajo en aplicaciones empresariales, con sus implicaciones en el acceso a datos y en la arquitectura limpia.
